`xyz2nlm`: Constructive Cartesian-to-Spherical 3D Harmonic-Oscillator Basis Transformation

1 Purpose

The package `xyz2nlm` constructs the spherical harmonic-oscillator basis

$$|N, L, M\rangle$$

from the Cartesian harmonic-oscillator basis

$$|n_x, n_y, n_z\rangle.$$

The shell quantum number is

$$N = n_x + n_y + n_z.$$

In the code this same shell quantum number is stored as metadata named `n`. Thus, throughout this document,

$$\texttt{state.n} = N.$$

The algorithm is constructive. It does not diagonalize angular-momentum matrices. Instead it performs two steps:

1. construct the highest-weight states
$$|N, L, L\rangle$$
directly in the Cartesian basis;

2. obtain the remaining states
$$|N, L, M\rangle, \qquad M < L,$$
by repeated application of the angular-momentum lowering operator
$$\hat{L}_- = \hat{L}_x - i\hat{L}_y.$$

The important implementation point is that all states live inside a fixed shell N, whose Cartesian basis dimension is

$$d_N = \frac{(N+1)(N+2)}{2}.$$

2 Cartesian basis indexing

For fixed N, define the set of Cartesian occupation configurations

$$\mathcal{C}_N = \left\{(n_x, n_y, n_z) \in \mathbb{N}_0^3 : n_x + n_y + n_z = N\right\}.$$

The number of such states is $d_N = (N+1)(N+2)/2$.

The implementation uses a bosonic permanent, or stars-and-bars, indexing map

$$J_N : \mathcal{C}_N \to \{0, 1, \ldots, d_N - 1\}.$$

This is the index convention used by the `mlx_create_c` package, following the bosonic occupation indexing described in Ref. [1]. The inverse operation is used conceptually as

$$J_N^{-1}(j) = (n_x, n_y, n_z).$$

A vector in the fixed-N Cartesian basis is therefore represented as

$$|\psi_N\rangle = \sum_{(n_x,n_y,n_z)\in\mathcal{C}_N} \psi_{n_x n_y n_z} \, |n_x, n_y, n_z\rangle,$$

or equivalently as a dense array

$$\psi[J_N(n_x, n_y, n_z)] = \psi_{n_x n_y n_z}.$$

The code generally stores only nonzero components. The compressed representation is

$$\big(\texttt{non\_zero\_ind}, \texttt{non\_zero\_val}\big),$$

where

- `non_zero_ind` is an integer `np.ndarray` containing the nonzero indices $J_N(n_x, n_y, n_z)$;
- `non_zero_val` is a complex `np.ndarray` containing the corresponding amplitudes.

Thus the compressed state means

$$|\psi_N\rangle = \sum_r \texttt{non\_zero\_val}[r] \; \big|J_N^{-1}\big(\texttt{non\_zero\_ind}[r]\big)\big\rangle \, .$$

3 State container classes

The relevant state containers are defined in

`xyz2nlm.custom_dataclasses`.

3.1 `TransientState`

The class

`xyz2nlm.custom_dataclasses.TransientState`

is used for intermediate calculations. It stores

`n`, `non_zero_ind`, `non_zero_val`.

The shell metadata `n` is required because the same integer index has meaning only after specifying the shell $N =$ `n`. The dense representation is recovered with

`state.as_ndarray()`.

This method creates a dense array of length

$$d_N = \frac{(N+1)(N+2)}{2}$$

and fills

`out[state.non_zero_ind]` = `state.non_zero_val`.

The class implements two algebraic operations.

Scaling. For $c \in \mathbb{C}$,

$$c|\psi\rangle$$

is represented by leaving `non_zero_ind` unchanged and replacing

$$\texttt{non\_zero\_val} \mapsto c\,\texttt{non\_zero\_val}.$$

Addition. For two states in the same shell N,

$$|\chi\rangle = |\psi\rangle + |\phi\rangle$$

is implemented by merging the two index sets and summing amplitudes on indices that occur in both inputs. In formula form,

$$\chi_j = \psi_j + \phi_j,$$

but the implementation performs this operation directly on the compressed index/value arrays.

3.2 `SphericalHOBasisState`

The class

`xyz2nlm.custom_dataclasses.SphericalHOBasisState`

stores a finalized spherical harmonic-oscillator state. It carries the same compressed vector data as `TransientState`, but also records

$$\texttt{n} = N, \qquad \texttt{l} = L, \qquad \texttt{m} = M.$$

It is not intended as the main arithmetic workhorse. Its role is to associate a compressed Cartesian vector with the spherical-basis label

$$|N, L, M\rangle.$$

It also supports extraction to a dense `np.ndarray` through

`state.as_ndarray()`.

4 Highest-weight seed states

4.1 Definition

The seed states are the highest-weight states

$$|N, L, L\rangle.$$

They are constructed directly in

`xyz2nlm.direct_seed_state_calculation.create_seed_state`.

The construction is based on the operator identity

$$|N, L, L\rangle \propto \left(R_{1,1,1}\right)^L \left(S^\dagger\right)^q |0, 0, 0\rangle, \qquad q = \frac{N - L}{2}.$$

The two creation-only operators are

$$R_{1,1,1} = -\frac{\ell}{2}\left(a_x^\dagger + i a_y^\dagger\right),$$

and

$$S^\dagger = a_x^{\dagger 2} + a_y^{\dagger 2} + a_z^{\dagger 2}.$$

Because both are polynomials only in creation operators, all factors commute. Therefore the expansion into Cartesian number states can be computed combinatorially.

The implementation does not literally apply these operators. It evaluates the closed-form Cartesian coefficients

$$|N, L, L\rangle = \sum_{n_x+n_y+n_z=N} C^{(N,L)}_{n_x,n_y,n_z} |n_x, n_y, n_z\rangle.$$

4.2 Admissible Cartesian configurations

A Cartesian state contributes to $|N, L, L\rangle$ only if

$$n_x + n_y + n_z = N,$$

$$n_z \text{ is even},$$

and

$$0 \le n_z \le N - L.$$

The parity condition

$$N - L \in 2\mathbb{Z}$$

is also required. Equivalently,

$$q = \frac{N-L}{2} \in \mathbb{N}_0.$$

For a valid configuration define

$$s = q - \frac{n_z}{2} = \frac{N - L - n_z}{2} = \frac{n_x + n_y - L}{2}.$$

4.3 The integer polynomial coefficient $S_{L,s}(n)$

The nontrivial cancellation in the seed-state coefficient is isolated in the integer coefficient

$$S_{L,s}(n).$$

It is defined by the generating function

$$(1+x)^L (1-x^2)^s = \sum_{n=0}^{L+2s} S_{L,s}(n) x^n.$$

Equivalently,

$$S_{L,s}(n) = \sum_\alpha (-1)^\alpha \binom{s}{\alpha} \binom{L}{n-2\alpha},$$

where the binomial coefficient is interpreted as zero when its lower argument is outside the allowed range.

The implementation computes $S_{L,s}(n)$ with Python's arbitrary-precision integers in

`xyz2nlm.direct_seed_state_calculation.Sls`.

It uses the recurrence

$$\boxed{S_{L,s}(n) = \frac{L}{n} S_{L,s}(n-1) - \frac{L+2s-n+2}{n} S_{L,s}(n-2)}$$

for

$$2 \le n \le L + 2s,$$

with initial values

$$S_{L,s}(0) = 1, \qquad S_{L,s}(1) = L.$$

For the special case $L = s = 0$, only $S_{0,0}(0) = 1$ is used.

This recurrence follows from

$$G_{L,s}(x) = (1+x)^L (1-x^2)^s$$

and

$$(1-x^2) G'_{L,s}(x) = [L - (L+2s)x]\, G_{L,s}(x).$$

Equating coefficients gives

$$n S_{L,s}(n) = L S_{L,s}(n-1) - (L + 2s - n + 2) S_{L,s}(n-2).$$

4.4 Closed-form seed coefficient

The coefficient of

$$|n_x, n_y, n_z\rangle$$

in $|N, L, L\rangle$ is

$$\boxed{C^{(N,L)}_{n_x,n_y,n_z} = (-1)^L i^{L-n_x} S_{L,s}(n_x) \binom{q}{s} \sqrt{2^{-L} \frac{(2L+1)!\,(L+q)!\,n_x!\,n_y!\,n_z!}{(L!)^2\, q!\,(N+L+1)!}}}$$

where

$$q = \frac{N-L}{2}, \qquad s = q - \frac{n_z}{2}.$$

The code uses the equivalent phase convention

$$(-1)^L i^{L-n_x} = i^{3L-n_x}.$$

Therefore the coefficient is evaluated in the form

$$C^{(N,L)}_{n_x,n_y,n_z} = \operatorname{sgn}(S_{L,s}(n_x))\, i^{3L-n_x} e^{\Phi},$$

where

$$\begin{aligned}
\Phi = {} & \log |S_{L,s}(n_x)| - \frac{L}{2} \log 2 \\
& + \log \Gamma(q+1) - \log \Gamma(s+1) - \log \Gamma(q-s+1) \\
& + \frac{1}{2} \Big[\log \Gamma(2L+2) + \log \Gamma(L+q+1) + \log \Gamma(n_x+1) + \log \Gamma(n_y+1) + \log \Gamma(n_z+1) \\
& \qquad - 2 \log \Gamma(L+1) - \log \Gamma(q+1) - \log \Gamma(N+L+2) \Big].
\end{aligned}$$

This is implemented in

`xyz2nlm.direct_seed_state_calculation.coefficient_seed_state_expansion`.

The logarithmic form is used to avoid overflow in factorials while still returning an ordinary double-precision complex coefficient.

The function

`xyz2nlm.direct_seed_state_calculation.create_seed_state`

does the full job:

1. generate all valid Cartesian configurations;
2. compute $S_{L,s}(n_x)$;
3. compute the coefficient $C^{(N,L)}_{n_x,n_y,n_z}$;
4. map configurations to indices $J_N(n_x, n_y, n_z)$;
5. package the result as a `xyz2nlm.custom_dataclasses.SphericalHOBasisState`.

5 Angular-momentum operators in the Cartesian basis

The angular-momentum operators are implemented in

`xyz2nlm.create_angular_momentum_matrices.CreateAngularMomentumMatrices`.

The implementation uses units with

$$\hbar = 1.$$

The Cartesian formulas used are

$$L_x = -i\, a_y^\dagger a_z + i\, a_z^\dagger a_y,$$

$$L_y = -i\, a_z^\dagger a_x + i\, a_x^\dagger a_z,$$

and

$$L_z = -i\, a_x^\dagger a_y + i\, a_y^\dagger a_x.$$

Therefore

$$\begin{aligned} L_+ &= L_x + iL_y \\ &= -i\, a_y^\dagger a_z + i\, a_z^\dagger a_y + a_z^\dagger a_x - a_x^\dagger a_z, \end{aligned}$$

and

$$\begin{aligned} L_- &= L_x - iL_y \\ &= -i\, a_y^\dagger a_z + i\, a_z^\dagger a_y - a_z^\dagger a_x + a_x^\dagger a_z. \end{aligned}$$

The method

`get_ladder_operators()`

returns

$$(L_+, L_-),$$

with L_- as the second output.

6 Sparse implementation of $a_i^\dagger a_j$

The angular-momentum operators are sums of terms of the form

$$c_{ij}\, a_i^\dagger a_j, \qquad i, j \in \{x, y, z\}.$$

The code constructs these terms without explicitly building dense matrices. It uses the following identity.

Let

$$\mathbf{k} = (k_x, k_y, k_z), \qquad k_x + k_y + k_z = N - 1.$$

Then

$$a_i^\dagger a_j |\mathbf{k} + \mathbf{e}_j\rangle = \sqrt{(k_i + 1)(k_j + 1)}\,|\mathbf{k} + \mathbf{e}_i\rangle.$$

Thus a term $c_{ij}a_i^\dagger a_j$ contributes

$$H_{J_N(\mathbf{k}+\mathbf{e}_i),\,J_N(\mathbf{k}+\mathbf{e}_j)} \mathrel{+}= c_{ij}\sqrt{(k_i+1)(k_j+1)}.$$

The core mapping procedure is:

```
def _calculate_mapping_matrices(self):
    self.Ns1 = mlx_create_c.create_bos_ns(self.N-1, 3)
    self.mapmat = mlx_create_c.create_mapmat_bos(self.Ns1)
    self.basis_size = self.mapmat[-1,-1] + 1

def _calculate_linear_operator_elements(self, i, j):
    mapmat = self.mapmat.view()
    Ns1 = self.Ns1.view()

    lhs_map = mapmat[:, i]
    rhs_map = mapmat[:, j]
    coefficients = np.sqrt((Ns1[:, i] + 1.) * (Ns1[:, j] + 1.))

    return lhs_map, rhs_map, coefficients
```

Here:

- `Ns1` stores all $(N-1)$-shell configurations $\mathbf{k}$;
- `mapmat[:, i]` stores
$$J_N(\mathbf{k}+\mathbf{e}_i)$$
for each $\mathbf{k}$;
- `mapmat[:, j]` stores
$$J_N(\mathbf{k}+\mathbf{e}_j);$$
- the coefficient array stores
$$\sqrt{(k_i+1)(k_j+1)}.$$

The resulting operators are packaged as

`scipy.sparse.linalg.LinearOperator`

instances with both `matvec` and `matmat` implemented. For an input vector ψ, a term $c_{ij}a_i^\dagger a_j$ acts as

$$(H\psi)_{J_N(\mathbf{k}+\mathbf{e}_i)} \mathrel{+}= c_{ij}\sqrt{(k_i+1)(k_j+1)}\,\psi_{J_N(\mathbf{k}+\mathbf{e}_j)}.$$

The same indexing rule is applied columnwise in `matmat`.

7 Generating $M < L$ states

After a seed state $|N, L, L\rangle$ is available, the remaining states in the same multiplet are generated by repeated application of L_-.

The angular-momentum normalization convention is

$$L_-|N, L, M\rangle = \sqrt{(L+M)(L-M+1)}\,|N, L, M-1\rangle.$$

Therefore

$$|N, L, M-1\rangle = \frac{L_-|N, L, M\rangle}{\sqrt{(L+M)(L-M+1)}}.$$

This is the normalization applied in

`BlockLmApplication.normalize_states_after_lm_application`.

If a matrix contains several current states as columns, the implementation uses `matmat` to apply L_- to all columns simultaneously, then divides each column by its own factor

$$\sqrt{(L+M)(L-M+1)}.$$

8 Block orchestration

The full basis generation is orchestrated by

`xyz2nlm.create_spherical_basis.SphericalHOBasis`.

The user initializes

`SphericalHOBasis(Nmax)`

and calls

`calculate_states()`.

The implementation allocates shell blocks for

$$0 \le N \le N_{\max}.$$

For each fixed N, the allowed angular momenta are

$$L = N, N-2, N-4, \ldots,$$

down to 0 for even N and down to 1 for odd N. Equivalently,

$$0 \le L \le N, \qquad N - L \in 2\mathbb{Z}.$$

The workflow is:

1. Create all highest-weight seed states

   $$|N, L, L\rangle$$

   using

   `create_seed_state(N,L)`.

   This step is done concurrently when possible.

2. Once all seed states for a fixed shell N are available, stack them as columns of a matrix

   $$B_N = \begin{bmatrix} |N, L_1, L_1\rangle & |N, L_2, L_2\rangle & \cdots \end{bmatrix}.$$

3. Obtain L_- for the same shell with

   `CreateAngularMomentumMatrices(N).get_ladder_operators()`.

4. Repeatedly apply L_- to the active block of columns, normalize each column by

   $$\sqrt{(L+M)(L-M+1)},$$

   and store the resulting states

   $$|N, L, M-1\rangle.$$

5. Drop a multiplet from the active block once its lowest state $M = -L$ has been produced.

This blockwise construction avoids applying L_- separately to every state. For each shell N, many states are updated simultaneously through the `LinearOperator.matmat` implementation.

9 Internal consistency checks

The mathematically expected checks are:

Normalization. For every produced state,

$$\langle N, L, M | N, L, M \rangle = 1.$$

L_z eigenvalue.

$$L_z |N, L, M\rangle = M |N, L, M\rangle.$$

Highest-weight condition. For seed states,

$$L_+ |N, L, L\rangle = 0.$$

Casimir eigenvalue.

$$L^2 |N, L, M\rangle = L(L+1) |N, L, M\rangle.$$

In code,

$$L^2 = L_x^2 + L_y^2 + L_z^2.$$

For numerical validation it is usually more stable to first check L_+ on highest-weight states and L_z on all states, because those checks involve less cancellation than a direct Cartesian-basis application of L^2.

10 Implementation summary for maintainers and LLMs

The code should be understood as follows:

- The package constructs a change-of-basis matrix from Cartesian harmonic-oscillator states to spherical harmonic-oscillator states.
- The shell quantum number N is called `n` in the code. A fixed-N Cartesian sector has dimension
$$(N+1)(N+2)/2.$$
- A state is stored sparsely by `non_zero_ind` and `non_zero_val`. The indices are occupation-state indices $J_N(n_x, n_y, n_z)$.
- `TransientState` is for arithmetic: scaling and addition.
- `SphericalHOBasisState` is for finalized basis states and carries the metadata (N, L, M).
- Seed states $|N, L, L\rangle$ are computed directly from a closed formula for
$$C^{(N,L)}_{n_x,n_y,n_z}.$$
No numerical diagonalization is involved.
- The integer factor $S_{L,s}(n_x)$ is computed exactly using Python integers and the recurrence
$$S_{L,s}(n) = \frac{L}{n} S_{L,s}(n-1) - \frac{L+2s-n+2}{n} S_{L,s}(n-2).$$
- The positive factorial prefactor in the coefficient is evaluated with `scipy.special.gammaln`.

- The states with $M < L$ are obtained by applying the sparse `LinearOperator` for L_- and normalizing by
$$\sqrt{(L+M)(L-M+1)}.$$
- The angular-momentum matrices are not formed densely. Each term $a_i^\dagger a_j$ is implemented by mapping
$$|\mathbf{k}+\mathbf{e}_j\rangle \mapsto |\mathbf{k}+\mathbf{e}_i\rangle$$
for all $(N-1)$-particle configurations $\mathbf{k}$.

References

[1] A. I. Streltsov, O. E. Alon, and L. S. Cederbaum, "General mapping for bosonic and fermionic operators in Fock space", Phys. Rev. A **81**, 022124 (2010). https://link.aps.org/doi/10.1103/PhysRevA.81.022124

[2] G. M. Koutentakis, `xyz2nlm`: Create the spherical harmonic oscillator basis from the Cartesian one. https://github.com/gkoutentakis/xyz2nlm

[3] G. M. Koutentakis, `mlx_create_c`: Port of the MATLAB create library in C for use in other programming languages. https://github.com/gkoutentakis/mlx_create_c